

Notebook

February 11, 2026

```
[ ]: import marimo as mo
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

1 Lab 4: Regression

1.1 Learning objectives

By the end of this lab, you should be able to: 1. Interpret regression as projecting onto a subspace / column space. 2. Fit and interpret simple linear regression. 3. Compare model flexibility (kNN, polynomial regression) and its impact on fit. 4. Compute and interpret training error (SSE) and discuss why it can be misleading.

2 Subspaces (warm-up linear algebra)

We will plot two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ in $\mathbb{R}^3$ and the **subspace they span**:

$$\text{span}(\mathbf{v}_1, \mathbf{v}_2) = \{ u \mathbf{v}_1 + v \mathbf{v}_2 \mid u, v \in \mathbb{R} \}.$$

```
[ ]: # Define two non-collinear vectors in R^3 (change these to experiment)
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
_u_vals = np.linspace(-20, 20, 81)
# Scalars for linear combinations u*v1 + v*v2
_v_vals = np.linspace(-20, 20, 81)
_u, _v = np.meshgrid(_u_vals, _v_vals)
_x = _u * v1[0] + _v * v2[0]
_y = _u * v1[1] + _v * v2[1]
# Coordinates of the spanned plane
_z = _u * v1[2] + _v * v2[2]
lim = 50
_fig = go.Figure(data=go.Surface(x=_x, y=_y, z=_z, colorscale='Viridis',
    opacity=0.6, showscale=False))
```

```

_fig.update_layout(scene=dict(xaxis_title='x', yaxis_title='y',
    ↪axis_title='z', xaxis_range=[-lim, lim], yaxis_range=[-lim, lim],
    ↪axis_range=[-lim, lim]), title='Subspace: span(v1, v2) = {u v1 + v v2}',
    ↪height=800, width=900)
_fig.add_trace(go.Scatter3d(x=[0, v1[0], None, 0, v2[0], None], y=[0, v1[1],
    ↪None, 0, v2[1], None], z=[0, v1[2], None, 0, v2[2], None], mode='lines',
    ↪line=dict(width=8, color='black'), name='v1 and v2')) # axis limits
_fig.add_trace(go.Scatter3d(x=[v1[0], v2[0]], y=[v1[1], v2[1]], z=[v1[2],
    ↪v2[2]], mode='text', text=['v1', 'v2'], showlegend=False))
# Surface representing span(v1, v2)
# Draw the two spanning vectors from the origin
# Label tips of vectors
_fig.show()

```

3 Advertising Sales Example (Regression)

```

[ ]: data = pd.read_csv('https://raw.githubusercontent.com/MMiDS-textbook/
    ↪MMiDS-textbook.github.io/refs/heads/main/utils/datasets/advertising.csv')
data.head()

```

```

[ ]: # Target (what we are trying to predict)
sales = data['sales'].to_numpy()

# Features / predictors
TV = data['TV'].to_numpy()
radio = data['radio'].to_numpy()
newspaper = data['newspaper'].to_numpy()

# Quick sanity checks
print('TV shape:', TV.shape, '| radio shape:', radio.shape, '| newspaper shape:
    ↪', newspaper.shape, '| sales shape:', sales.shape)

```

TV shape: (200,) | radio shape: (200,) | newspaper shape: (200,) | sales shape: (200,)

TODO Question (interpret the scatter plots):

We will plot each feature individually against sales.

- Which variable(s) look **strongly** associated with sales?
 - TV and Radio
- Which variable(s) look **weakly** associated?
 - Newspaper

```

[ ]: _fig, _ax = plt.subplots(1, 3, figsize=(18, 5), sharey=True)

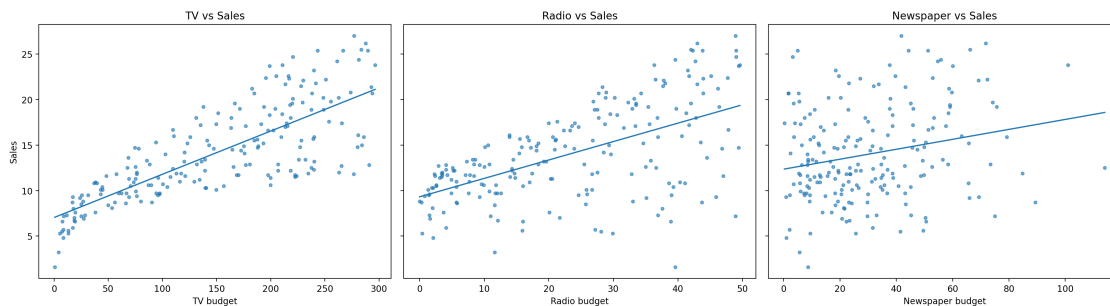
# Helper: add a simple best-fit line for visual guidance (not the full
    ↪multivariate model)

```

```

def _add_line(x, y, axis):
    m, b = np.polyfit(x, y, 1)
    xs = np.linspace(x.min(), x.max(), 200)
    axis.plot(xs, m * xs + b)
_ax[0].scatter(TV, sales, s=12, alpha=0.6)
_add_line(TV, sales, _ax[0])
_ax[0].set_xlabel('TV budget')
_ax[0].set_ylabel('Sales')
_ax[0].set_title('TV vs Sales')
_ax[1].scatter(radio, sales, s=12, alpha=0.6)
_add_line(radio, sales, _ax[1])
_ax[1].set_xlabel('Radio budget')
_ax[1].set_title('Radio vs Sales')
_ax[2].scatter(newspaper, sales, s=12, alpha=0.6)
_add_line(newspaper, sales, _ax[2])
_ax[2].set_xlabel('Newspaper budget')
_ax[2].set_title('Newspaper vs Sales')
plt.tight_layout()
plt.show()

```



From the plots: - **TV** and **radio** show a clear positive relationship with sales. - **Newspaper** looks weaker (more scatter; the best-fit line is flatter).

Important: These are *one-feature-at-a-time* views. A variable can look weak alone, but still matter when combined with other features (and vice versa).

```

[ ]: _X = data[['TV', 'radio']]
     _y = data['sales']
     _X_train, _X_test, _y_train, _y_test = train_test_split(_X, _y, test_size=0.2,
     ↪ random_state=42)
     _model = LinearRegression()
     _model.fit(_X_train, _y_train)
     _coefs = _model.coef_
     _inter = _model.intercept_
     print('Fitted model: sales = {:.3f} + ({:.3f})*TV + ({:.3f})*Radio'.
     ↪ format(_inter, _coefs[0], _coefs[1]))

```

```

_tv_vals = np.linspace(data['TV'].min(), data['TV'].max(), 50)
_radio_vals = np.linspace(data['radio'].min(), data['radio'].max(), 50)
_TV_mesh, _RADIO_mesh = np.meshgrid(_tv_vals, _radio_vals)
_SALES_mesh = _inter + _coefs[0] * _TV_mesh + _coefs[1] * _RADIO_mesh
_fig = go.Figure(data=go.Surface(x=_RADIO_mesh, y=_TV_mesh, z=_SALES_mesh,
    ↪ colorscale='Viridis', opacity=0.6))
_fig.update_layout(scene=dict(xaxis_title='Radio', yaxis_title='TV',
    ↪ xaxis_title='Sales', xaxis_range=[data['radio'].min(), data['radio'].max()],
    ↪ yaxis_range=[data['TV'].min(), data['TV'].max()], zaxis_range=[data['sales'].
    ↪ min(), data['sales'].max()]), title='Regression Hyperplane for Sales
    ↪ Prediction', height=1000, width=1000)
_fig.add_trace(go.Scatter3d(x=data['radio'], y=data['TV'], z=data['sales'],
    ↪ mode='markers', marker=dict(size=4, color='black', opacity=1)))
_fig.show()

```

Fitted model: $\text{sales} = 3.028 + (0.045) \cdot \text{TV} + (0.191) \cdot \text{Radio}$

Now compare **TV + newspaper** in a multivariate model.

```

[ ]: _X = data[['TV', 'newspaper']]
_y = data['sales']
_X_train, _X_test, _y_train, _y_test = train_test_split(_X, _y, test_size=0.2,
    ↪ random_state=42)
_model = LinearRegression()
_model.fit(_X_train, _y_train)
_coefs = _model.coef_
_inter = _model.intercept_
print('Fitted model: sales = {:.3f} + ({:.3f})*TV + ({:.3f})*Newspaper'.
    ↪ format(_inter, _coefs[0], _coefs[1]))
_tv_vals = np.linspace(data['TV'].min(), data['TV'].max(), 50)
_newspaper_vals = np.linspace(data['newspaper'].min(), data['newspaper'].max(),
    ↪ 50)
_TV_mesh, _NEWSPAPER_mesh = np.meshgrid(_tv_vals, _newspaper_vals)
_SALES_mesh = _inter + _coefs[0] * _TV_mesh + _coefs[1] * _NEWSPAPER_mesh
_fig = go.Figure(data=go.Surface(x=_NEWSPAPER_mesh, y=_TV_mesh, z=_SALES_mesh,
    ↪ colorscale='Viridis', opacity=0.6))
_fig.update_layout(scene=dict(xaxis_title='Newspaper', yaxis_title='TV',
    ↪ xaxis_title='Sales', xaxis_range=[data['newspaper'].min(), data['newspaper'].
    ↪ max()], yaxis_range=[data['TV'].min(), data['TV'].max()],
    ↪ zaxis_range=[data['sales'].min(), data['sales'].max()]), title='Regression
    ↪ Hyperplane for Sales Prediction', height=1000, width=1000)
_fig.add_trace(go.Scatter3d(x=data['newspaper'], y=data['TV'], z=data['sales'],
    ↪ mode='markers', marker=dict(size=4, color='black', opacity=1)))
_fig.show()

```

Fitted model: $\text{sales} = 5.479 + (0.046) \cdot \text{TV} + (0.056) \cdot \text{Newspaper}$

In this fitted plane, you may see cases where: - holding TV fixed, higher newspaper can correspond to slightly lower predicted sales (a **negative coefficient**), - or the effect may be near zero.

That does **not** mean “newspaper reduces sales in real life.” It means: *given this dataset and model*, newspaper does not add strong predictive signal in the same direction as TV/radio.

4 K-Nearest Neighbors (kNN) Regression (intuition for model flexibility)

```
[ ]: def knnregression(x, y, k, xnew):
    """1D kNN regression.

    Parameters
    -----
    x : array-like, shape (n,)
        Input feature values.
    y : array-like, shape (n,)
        Targets.
    k : int
        Number of neighbors.
    xnew : float
        Query point.

    Returns
    -----
    yhat : float
        kNN prediction at xnew.
    """
    _n = len(x)
    closest = np.argsort([abs(x[i] - xnew) for i in range(_n)]) #Sort indices
    ↪by distance to xnew
    return np.mean(y[closest[:k]])
```

TODO Question: As k increases, what happens to the kNN curve?

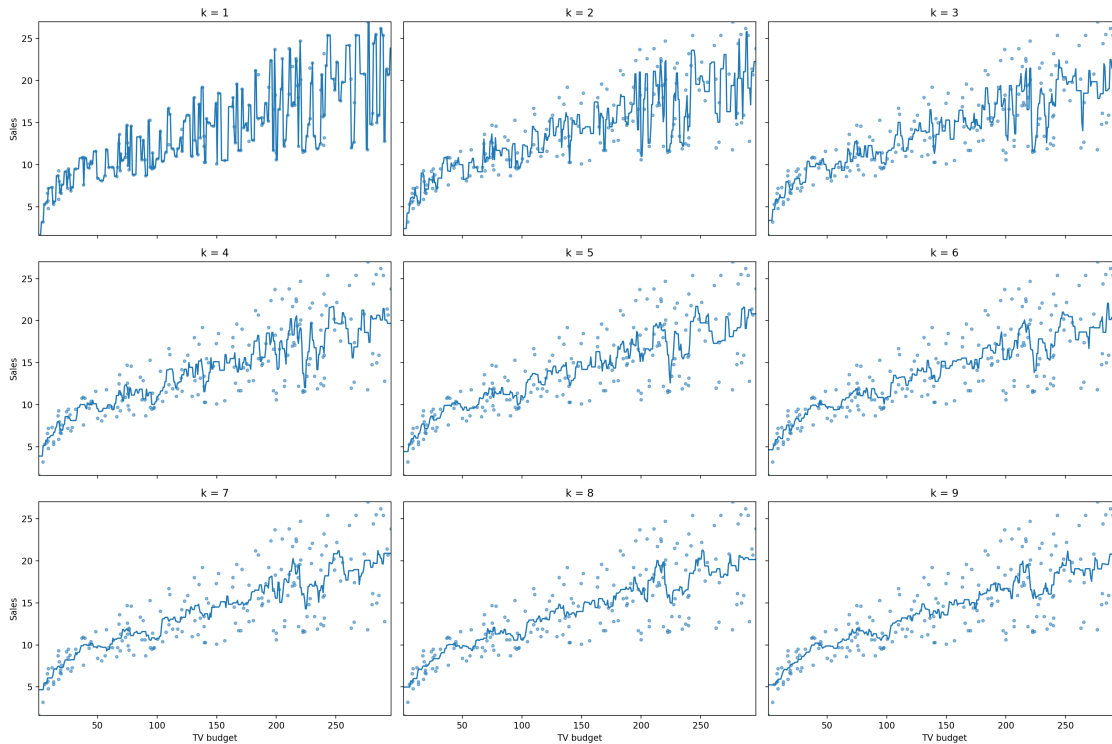
Track two things: The curve becomes smoother and the model begins to underfit the data as it uses a higher volume of points to determine the local average.

```
[ ]: _fig, _ax = plt.subplots(3, 3, figsize=(18, 12), sharex=True, sharey=True)
    _xgrid = np.linspace(TV.min(), TV.max(), num=400)
    xmin, xmax = (TV.min(), TV.max())
    ymin, ymax = (sales.min(), sales.max())
    for idx, _k in enumerate(range(1, 10)):
        r, c = divmod(idx, 3)
        _ax[r, c].scatter(TV, sales, s=10, alpha=0.5)
        _yhat = [knnregression(TV, sales, _k, xnew) for xnew in _xgrid]
        _ax[r, c].plot(_xgrid, _yhat)
        _ax[r, c].set_title(f'k = {_k}')
        _ax[r, c].set_xlim(xmin, xmax)
        _ax[r, c].set_ylim(ymin, ymax)
```

```

for a in _ax[-1, :]:
    a.set_xlabel('TV budget')
for a in _ax[:, 0]:
    a.set_ylabel('Sales')
plt.tight_layout()
plt.show()

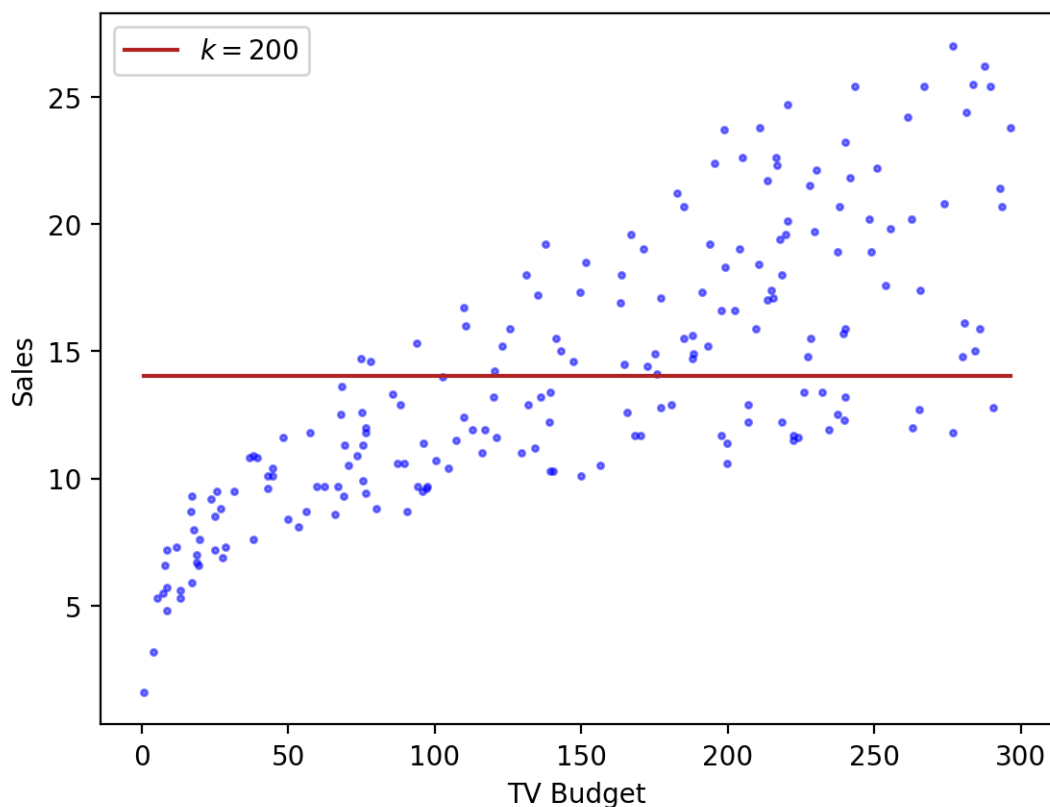
```



```

[ ]: _k = TV.shape[0]
plt.scatter(TV, sales, s=5, c='b', alpha=0.5)
_xgrid = np.linspace(TV.min(), TV.max(), num=1000)
_yhat = [knnregression(TV, sales, _k, xnew) for xnew in _xgrid]
plt.plot(_xgrid, _yhat, color='firebrick', label=f'$k = {_k}$')
plt.legend()
plt.xlabel('TV Budget')
plt.ylabel('Sales')
plt.show()

```



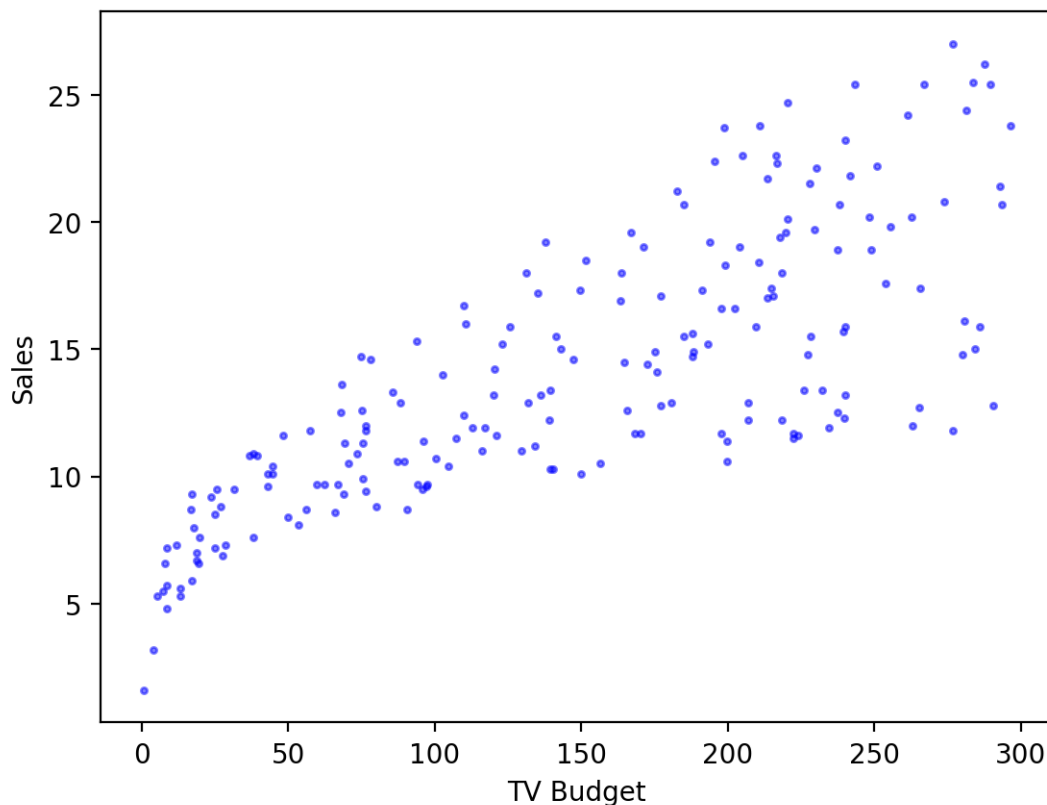
5 Ordinary Least Squares (OLS) Regression (single feature)

Now we switch to a *parametric* model: a line

$$\hat{y} = \beta_0 + \beta_1 x. \quad (1)$$

We'll: 1. build the **design matrix** X , 2. derive β using the closed-form solution, 3. interpret the coefficients and compute training error.

```
[ ]: # we have TV and Sales, we want to use TV to predict sales
      # begin by plotting them again
      plt.scatter(TV, sales, s=5, c='b', alpha=0.5)
      plt.xlabel('TV Budget')
      plt.ylabel('Sales')
      plt.show()
```



```
[ ]: _n = TV.shape[0]
```

```
[ ]: # we need to standardize the TV column (column 1)

# TODO: Standardize X[:, 1] to have mean 0 and std 1
_tv_std = (TV - np.mean(TV)) / np.std(TV)
# X[:, 1] = ...

# TODO: Display the first few rows to confirm standardization worked
X_1 = np.column_stack([np.ones(len(TV)), _tv_std])
X_1[:5, :]
```

TODO Question: Why do we include a first column of 1's in X ?

Closed-form OLS solution:

$$\beta = (X^T X)^{-1} X^T y. \quad (2)$$

Answer: The first column of 1s in X are for β_0 . The y-intercept is not 0 so the line doesn't pass through the origin as that wouldn't be realistic.

```
[ ]: # Now we can train our model using Ordinary Least Squares (closed form)
```



```

# TODO: Compute betas using the closed-form OLS solution
#  $\text{betas} = (X^T X)^{-1} X^T y$ 
betas_1 = np.linalg.inv(X_1.T @ X_1) @ X_1.T @ sales
# betas = ...

# TODO: Print betas
print(betas_1)

```

```
[14.0225      4.07100612]
```

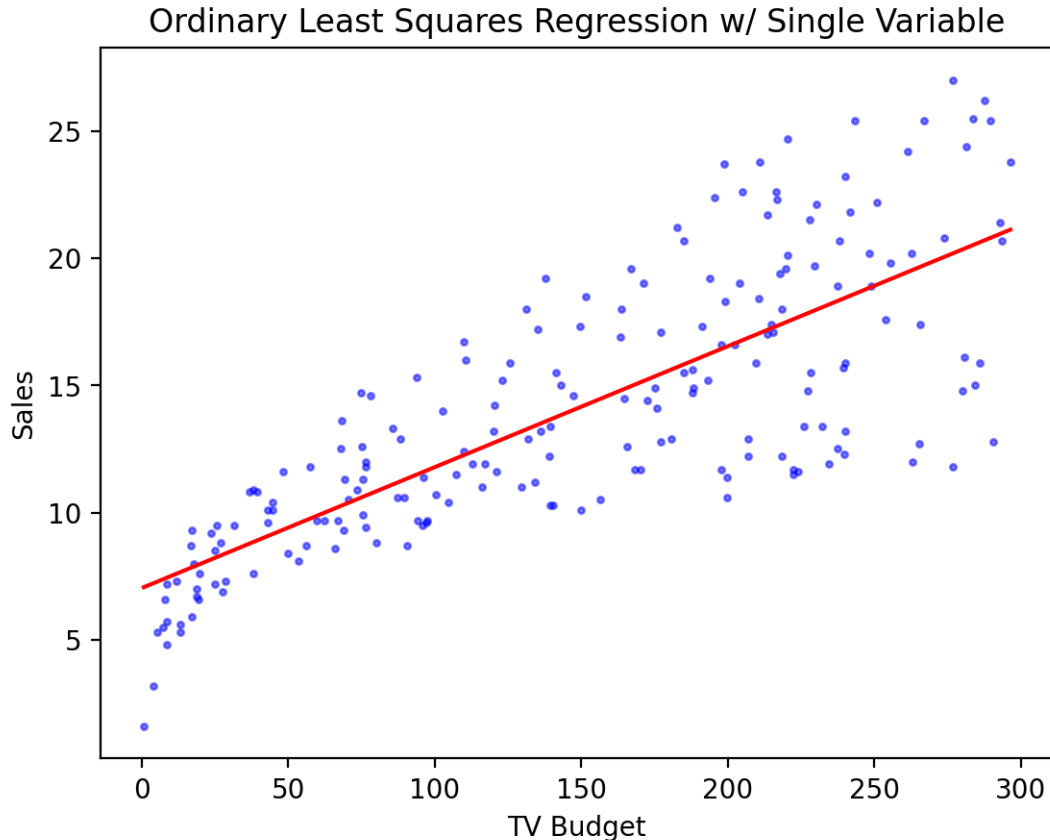
TODO Question: What does the first component of the beta represents in regards to a line? What about the second component?

The first component of beta (β_0) is the y-intercept. The second component of beta (β_1) is the slope.

```

[ ]: _TV_vals = np.linspace(X_1[:, 1].min(), X_1[:, 1].max(), num=1000)
plt.scatter(TV, sales, s=5, c='b', alpha=0.5)
plt.plot(_TV_vals * np.std(TV) + np.mean(TV), betas_1[0] + betas_1[1] * _TV_vals, 'r')
plt.ylabel('Sales')
plt.xlabel('TV Budget')
plt.title('Ordinary Least Squares Regression w/ Single Variable')
plt.show()

```



In this lab we'll start with **training SSE**:

$$\text{SSE} = \sum_i (y_i - \hat{y}_i)^2. \quad (3)$$

```
[ ]: # Sum of Squared Errors (training)
# TODO: Compute predictions on the training data
_preds = X_1 @ betas_1
# preds = ...
# TODO: Compute training SSE
# TODO: Print m1_sse
m1_sse = np.sum((sales - _preds) ** 2)
print(m1_sse)
```

2102.5305831313512

6 Polynomial Regression

We increase model flexibility by adding polynomial terms. A degree-2 model is:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2.$$

Key idea: more flexibility almost always lowers training SSE, but can overfit.

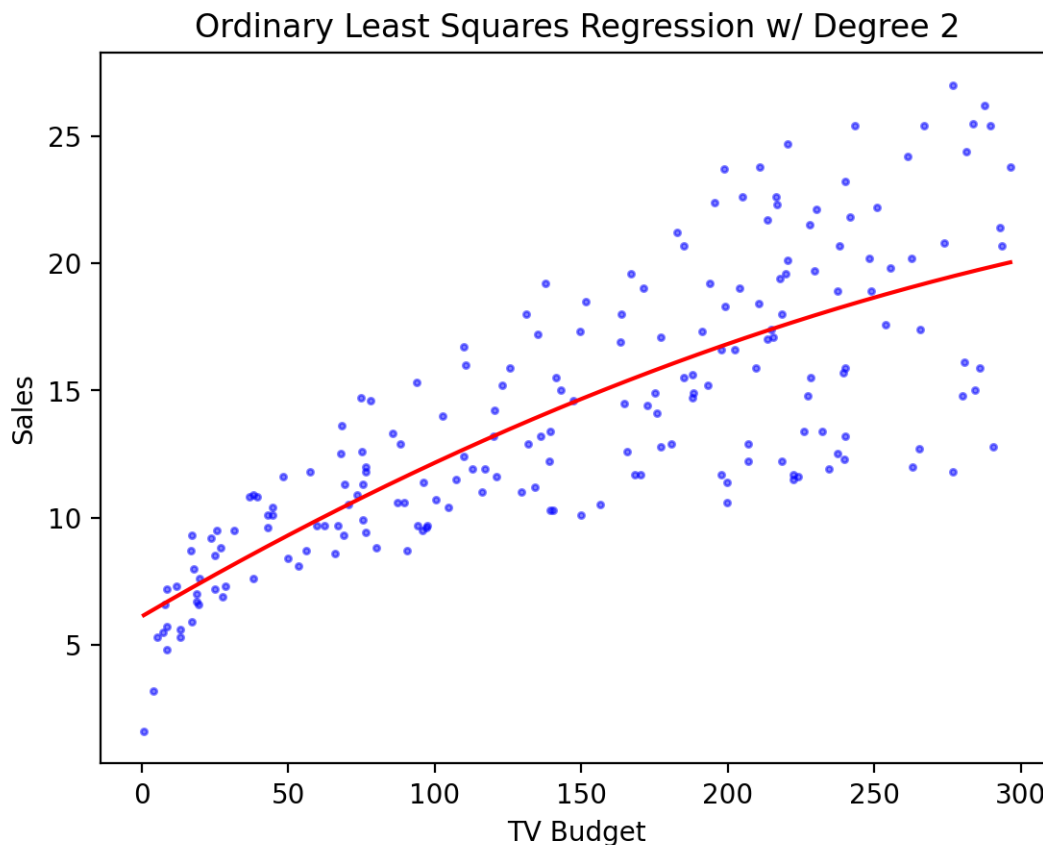
Increase complexity by adding polynomial terms (here, a squared term).

```
[ ]: # now we may want a more complex model with more freedom
# Let's add a squared term of TV to the design matrix.
# TODO: Build a new design matrix for a degree-2 polynomial model.
# Recommended approach:
X_2 = np.column_stack([X_1, X_1[:, 1] ** 2])
```

```
[ ]: # Using our new design matrix, let's fit the degree-2 model with OLS

# TODO: Compute betas for the degree-2 model
betas_2 = np.linalg.inv(X_2.T @ X_2) @ X_2.T @ sales
# betas = ...
```

```
[ ]: _TV_vals = np.linspace(X_2[:, 1].min(), X_2[:, 1].max(), num=1000)
plt.scatter(TV, sales, s=5, c='b', alpha=0.5)
plt.plot(_TV_vals * np.std(TV) + np.mean(TV), betas_2[0] + betas_2[1] * _TV_vals + betas_2[2] * _TV_vals ** 2, 'r')
plt.ylabel('Sales')
plt.xlabel('TV Budget')
plt.title('Ordinary Least Squares Regression w/ Degree 2')
plt.show()
```



TODO Question: Compare degree-1 vs degree-2 fits.

Look for: - Does the curve capture structure that the line missed? - Does it start bending to chase noise?

Answer: The degree-2 fit captures the diminishing returns of TV advertising as budget increases which the degree-1 fit missed. The curve is still relatively simple and smooth, so it does not seem to be bending to chase noise.

```
[ ]: # Compare training SSE for the degree-2 model

# TODO: Compute predictions and SSE for the degree-2 model
_preds = X_2 @ betas_2
m2_sse = np.sum((sales - _preds) ** 2)
# preds = ...
# m2_sse = ...

# TODO: Print m2_sse
print(m2_sse)
```

2063.732606050492

7 Multiple OLS Regression

Now we include multiple features at once:

$$\hat{y} = \beta_0 + \beta_1 TV + \beta_2 radio + \beta_3 newspaper.$$

The same closed-form solution applies; only the design matrix changes.

We cannot fully visualize

$$\mathbb{R}^3 \rightarrow \mathbb{R} \quad (4)$$

\end{equation}

(3 features to 1 target) in a single static plot.

Instead, we can: - look at **slices** (hold one variable constant), - or evaluate using metrics (SSE / test error).

```
[ ]: # remember our data?
data.head(2)
```

```
[ ]: _features = data[['TV', 'radio', 'newspaper']].to_numpy()
_features_std = (_features - _features.mean(axis=0)) / _features.std(axis=0)
_n = _features_std.shape[0]
X_3 = np.column_stack([np.ones(_n), _features_std])
print('Design matrix X shape:', X_3.shape)
X_3[:5, :]
```

Design matrix X shape: (200, 4)

```
[ ]: # Train multivariate OLS and compute training SSE

# TODO: Fit multivariate OLS
betas_3 = np.linalg.inv(X_3.T @ X_3) @ X_3.T @ sales
# betas = ...

# TODO: Predict on training data
_preds = X_3 @ betas_3
# preds = ...

# TODO: Compute training SSE
m3_sse = np.sum((sales - _preds) ** 2)
# m3_sse = ...

print('Model 1 (linear TV only) SSE:', m1_sse)
print('Model 2 (quadratic TV) SSE:', m2_sse)
print('Model 3 (TV + radio + newspaper) SSE:', m3_sse)
print('\nBetas (intercept, TV, radio, newspaper) on standardized features:')
print(betas_3)
```

Model 1 (linear TV only) SSE: 2102.5305831313512
Model 2 (quadratic TV) SSE : 2063.732606050492
Model 3 (TV + radio + newspaper) SSE: 556.8252629021872

Betas (intercept, TV, radio, newspaper) on standardized features:
[14.0225 3.91925365 2.79206274 -0.02253861]

TODO Question: What happened to our SSE after using all variables available (TV, newspaper, and radio) to predict sales?

Answer: The SSE decreased greatly compared to the single-variable models when we used all variables available to predict sales.

Note, we cannot plot this multi-dimensional space. Instead, we can only plot it from one view. We may keep the other values constant and change only one, seeing how that variable effects the predictions.